Anti-Bluff Mandate Reinforcement (8th invocation response) — Design

Date: 2026-05-05 **Spec ID:** Group A of the 2026-05-05 multi-group brainstorm **Status:** Approved (user said "all good" 2026-05-05) **Implementation skill:** `writing-plans` (next)

Forensic anchor

The operator has now invoked the anti-bluff mandate **EIGHT times across two working days**. The most recent invocation, verbatim:

> "all existing tests and Challenges do work in anti-bluff manner — they MUST confirm that all tested codebase really works as expected! We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can't be used! This MUST NOT be the case and execution of tests and Challenges MUST guarantee the quality, the completion and full usability by end users of the product!"

Concurrent finding: a 7-day-old architectural bluff in `submodules/containers/pkg/emulator/Boot()` (hardcoded `ADBPort=5555`) was discovered today via ultrathink-driven systematic-debugging. The matrix runner had been silently testing the FIRST AVD's emulator N times in every multi-AVD run, while the attestation file falsely recorded N rows for N different AVDs. See `.lava-ci-evidence/sixth-law-incidents/2026-05-05-matrix-runner-port-collision-bluff.json`.

The 8th invocation + the fresh architectural-bluff discovery jointly require:

1. Constitutional acknowledgment of the 8th invocation
2. Tightened bluff-hunt cadence (so the next architectural bluff is caught faster)
3. Production-code coverage in bluff hunts (so non-test bluffs are catchable at all)
4. Pre-push hook enforcement of the new cadence (deferred to Group A-prime spec)

This spec covers (1)–(3) as constitutional doc updates. (4) is documented as `6.N-debt` with a forward-link to Group A-prime.

Goals

- Mechanical-record acknowledgment of the 8th anti-bluff mandate invocation in `CLAUDE.md` §6.L

- Codify the tightened bluff-hunt cadence in Seventh Law clause 5 + new clause §6.N
- Codify production-code coverage in bluff hunts (was previously implicit; now explicit)
- Propagate the new clause across all consuming submodule + service docs (~20 files), matching the §6.M pattern from commit `20539d5`
- Land the 2026-05-05 phase-end bluff-hunt evidence as a JSON file
- Land an audit attestation verifying the propagation completed

Non-goals

- **Pre-push hook code changes** — owed via Group A-prime spec; this spec only documents the requirement as `6.N-debt`
- **Substantive constitutional change beyond cadence + production-code coverage** — count bump + new clause only; no rewrites of §6.J/§6.L/Seventh Law clauses 1–4 + 6–7
- **New constitution-checker code** — `scripts/check-constitution.sh` continues to function unchanged this round
- **Build/test runs** — no production code touched; no rebuild required

Architecture

Four artifact categories produced by this spec:

```
A. Constitutional doc updates
   ├── root CLAUDE.md       → §6.L count + Seventh Law clause 5 +
new §6.N + 6.N-debt entry
   ├── root AGENTS.md       → §6.N bullet
   ├── lava-api-go × 3      → §6.N reference block in CLAUDE.md,
AGENTS.md, CONSTITUTION.md
   └── 16 × submodules/*/CLAUDE.md → §6.N reference block
(Containers gets stronger variant)

B. Bluff-hunt evidence file
   └── .lava-ci-evidence/bluff-hunt/2026-05-05.json

C. Anti-bluff mandate audit attestation
   └── .lava-ci-evidence/anti-bluff-audit-2026-05-05.json

D. §6.N-debt placeholder (inside root CLAUDE.md)
   └── Forward-references the upcoming Group A-prime spec for pre-
push hook code
```

No new files outside `.lava-ci-evidence/` and `docs/superpowers/specs/`. No code, no schema changes.

Constitutional content

A.1 Root `CLAUDE.md` § 6.L count bump

Change the count from `SEVEN TIMES` to `EIGHT TIMES` and append the verbatim 2026-05-05 mandate text to the existing parenthetical-list of invocations. Format follows the existing pattern; the parenthetical is the load-bearing forensic record.

A.2 Root `CLAUDE.md` Seventh Law clause 5 expansion

Add two subsections to clause 5 (Recurring Bluff Hunt), keeping the existing per-phase 5-target rule as the baseline:

- **5.a — Cadence tightening** — cross-reference §6.N.1 (defined below). Three triggers for in-cycle bluff hunts beyond the 2–4 week phase-end cadence:
  1. Per operator anti-bluff-mandate invocation (lighter scope after first/day)
  2. Per matrix-runner / gate change (pre-push enforced — owed via Group A-prime)
  3. Per phase-gating attestation file added (pre-push enforced — owed via Group A-prime)
- **5.b — Production-code coverage** — cross-reference §6.N.2. Bluff hunts MUST sample production code beyond `*Test.kt` files. Layered scope:
  1. Mandatory: 2 files per phase from gate-shaping code (scripts/tag.sh helpers, scripts/check-constitution.sh, submodules/containers/pkg/emulator, cmd/emulator-matrix, matrix.go writeAttestation)
  2. Recommended: 0–2 additional from broader CI-touched code (anything invoked by `scripts/ci.sh` or `scripts/run-emulator-tests.sh`)
  3. Conceptual filter: "would a bug here be invisible to existing tests?" — prefer files that, if buggy, would silently fool the gate

A.3 Root `CLAUDE.md` new §6.N

Title: **"Bluff-Hunt Cadence Tightening + Production Code Coverage (added 2026-05-05)"**

Forensic anchor: the 7-day-old architectural bluff in `submodules/containers/pkg/emulator/Boot()` exposed by ultrathink-driven systematic-debugging on 2026-05-05; recorded at `.lava-ci-evidence/sixth-law-incidents/2026-05-05-matrix-runner-port-collision-bluff.json`. The bluff was invisible to all existing `*Test.kt`-targeted bluff hunts because the buggy code was production Go code that no test could mutate-rehearse — only end-to-end multi-AVD matrix runs would have caught it, and those runs were themselves rendered green by the bluff (which is the textbook clause-6.J failure mode).

Three sub-rules + a debt clause:

6.N.1 — Tightened cadence (formerly clause 5's 2–4-week-only rule)

The 2–4 week phase-end cycle remains the baseline. Three additional triggers fire in-cycle:

1. **Per operator anti-bluff-mandate invocation.** First invocation in any 24h window: full 5+2 hunt (5 `*Test.kt` per existing clause 5 + 2 production-code files per §6.N.2). Subsequent invocations within the same 24h: lighter "incident-response hunt" — 1–2 files most relevant to the invocation context (e.g., the area of code the latest discovery flagged).
2. **Per matrix-runner / gate change** (pre-push enforced — owed via Group A-prime). Any commit that touches `submodules/containers/pkg/emulator/`, `scripts/run-emulator-tests.sh`, `scripts/tag.sh`, or `scripts/check-constitution.sh` MUST be accompanied by a 1-target falsifiability rehearsal in the area of change. The rehearsal goes in the commit body (Bluff-Audit stamp) and the changed-area file SHOULD be one of the production-code targets.
3. **Per phase-gating attestation file added** (pre-push enforced — owed via Group A-prime). Any new file under `.lava-ci-evidence/sp3a-challenges/`, `.lava-ci-evidence/<tag>/real-device-verification.md` or `.json`, or `.lava-ci-evidence/sixth-law-incidents/` MUST be accompanied by a falsifiability rehearsal of the underlying production code path the attestation claims to cover. The rehearsal evidence MAY be the same file (rehearsal embedded in the attestation) OR a separate companion file in the same dir.

6.N.2 — Production-code coverage in bluff hunts

Bluff hunts MUST sample production code, not just `*Test.kt` / `*_test.go`. Layered:

- **Mandatory minimum (per phase):** 2 files from gate-shaping production code. Gate-shaping = files whose output determines pass/fail of a constitutional gate. Canonical list: `scripts/tag.sh` helpers, `scripts/check-constitution.sh`, `scripts/bluff-hunt.sh`, `submodules/containers/pkg/emulator/`, `submodules/containers/cmd/emulator-matrix/`, the matrix runner's writeAttestation function. The list grows as new gate-shaping code lands.
- **Recommended additional (per phase):** 0–2 files from broader CI-touched code — anything invoked by `scripts/ci.sh` or `scripts/run-emulator-tests.sh`, including Lava-domain Kotlin/Go production paths.
- **Conceptual filter:** for each candidate file, ask "would a bug here be invisible to existing tests?" Prefer files where the answer is yes — those are the bluff-rich targets.

The mutation-rehearsal protocol from clause 5 applies unchanged: pick file → apply deliberate mutation that affects the gate's verdict → run the gate → confirm the gate fails (or surfaces the wrong outcome) → revert → re-run → confirm green again. Record outcome in `.lava-ci-evidence/bluff-hunt/<date>.json`.

6.N.3 — §6.N-debt (forward-reference to Group A-prime spec)

The pre-push hook enforcement clauses (6.N.1.2 + 6.N.1.3 above) are documented but NOT yet implemented. Implementation is deferred to Group A-prime spec, which this Group A spec spawns as the next brainstorming + writing-plans cycle. Until Group A-prime ships:

- 6.N.1.1 (per-invocation hunt) is operator-driven manual cadence

- 6.N.1.2 + 6.N.1.3 are documented requirements only — no mechanical enforcement
- The §6.N-debt entry stays open in `CLAUDE.md` until Group A-prime closes it

Inheritance — clause 6.N applies recursively to every submodule, every feature, and every new artifact. Submodule constitutions MAY add stricter cadence requirements (e.g., Containers SHOULD bluff-hunt every change to `pkg/emulator/` since it's the source of truth for the matrix gate) but MUST NOT relax this clause.

A.4 Propagation pattern

Matches the §6.M propagation from commit `20539d5`. Each target file gets a "Clause 6.N (added 2026-05-05, inherited per 6.F)" block:

target	block flavor
root `AGENTS.md`	one-line bullet alongside existing 6.G/H/I/J/K/L/M bullets, summarizing 6.N.1+6.N.2
`lava-api-go/CLAUDE.md`	full reference block, ends-of-file matching existing 6.M placement
`lava-api-go/AGENTS.md`	reference block under "Host Machine Stability Directive" section (following 6.M's placement)
`lava-api-go/CONSTITUTION.md`	reference block in "Inherited rules" section
16 × `submodules/*/CLAUDE.md`	reference block at end of file, mirroring §6.M placement
`submodules/containers/CLAUDE.md`	**stronger variant** — Containers is the source of truth for matrix-runner/gate code; 6.N-Containers variant explicitly mandates bluff-rehearsal on every `pkg/emulator/` change (stricter than the parent rule)

The `app/CLAUDE.md`, `core/CLAUDE.md`, `feature/CLAUDE.md` scoped docs are NOT touched — they reference the root constitution and don't carry per-clause inheritance blocks.

Evidence files

B. `.lava-ci-evidence/bluff-hunt/2026-05-05.json`

Documents this round's phase-end bluff hunt as the architectural-bluff discovery. Schema:

```
{
  "date": "2026-05-05",
  "protocol": "Seventh Law clause 5 (Recurring Bluff Hunt) + new
        clause 6.N.1 + 6.N.2",
  "session_context":
        "8th anti-bluff invocation; ultrathink-driven systematic-
        debugging session",
```

```
  "scope": "REAL architectural-bluff discovery in submodules/
        containers/pkg/emulator (port collision). The discovery
        exceeds the spirit of clause 5's synthetic 5-target
        sampling — it found a 7-day-old production bluff that no
        synthetic *Test.kt mutation could have caught.",
  "primary_finding": {
    "target": "submodules/containers/pkg/emulator/
        android.go::Boot",
    "bluff_classification": "Hardcoded-port multi-target test
        runner",
    "remediation_commit": "submodules/containers commit 648a4bb
        (dynamic port discovery) + f6d09cb (Teardown wait-for-
        exit)",
    "incident_record": ".lava-ci-evidence/sixth-law-incidents/
        2026-05-05-matrix-runner-port-collision-bluff.json"
  },
  "satisfies_clause_5_5plus2": "yes — the architectural finding
        takes precedence over synthetic sampling per the new 6.N.2
        conceptual filter",
  "synthetic_sampling_NOT_done_this_round": "by design — the
        architectural finding is more valuable. Listed
        transparently for next-phase reference: the 5+2 sampling
        that would have been done would target X *Test.kt files +
        Y production files chosen per 6.N.2's gate-shaping list."
}
```

C. `.lava-ci-evidence/anti-bluff-audit-2026-05-05.json`

Per-file post-update verification. Schema:

```
{
  "date": "2026-05-05",
  "purpose": "Verify §6.N propagation across all 20 target files
        completed",
  "audit_method": "grep for '6\
        \.N' references in each target file post-commit",
  "results": {
    "root/CLAUDE.md": { "expected_refs": ">= 5", "found":
        "<count>" },
    "root/AGENTS.md": { "expected_refs": ">= 1", "found":
        "<count>" },
    "lava-api-go/CLAUDE.md": { "expected_refs": ">= 1", "found":
        "<count>" },
    "lava-api-go/AGENTS.md": { "expected_refs": ">= 1", "found":
        "<count>" },
    "lava-api-go/CONSTITUTION.md": { "expected_refs": ">= 1",
        "found": "<count>" },
    "submodules/auth/CLAUDE.md": { "expected_refs": ">= 1",
        "found": "<count>" },
    "...": "..."
  },
  "gaps_remediated": [],
  "summary": {
    "files_checked": 23,
    "files_with_required_refs": 23,
```

```
    "files_with_gaps_remediated_in_this_audit": 0
  }
}
```

The audit file is generated AFTER the doc commits land. If any gap is found, the audit fixes it inline and re-runs.

Verification

Doc-only spec, so verification is grep-based:

1. `grep -rE "6\.N" CLAUDE.md AGENTS.md` → at least 5 hits in CLAUDE.md, 1 in AGENTS.md
2. `for f in submodules/*/CLAUDE.md lava-api-go/*.md; do grep -c "6\.N" "$f"; done` → every count ≥ 1
3. `grep -E "EIGHT TIMES" CLAUDE.md` → exactly 1 hit (the bumped count)
4. `grep -c "6.N-debt" CLAUDE.md` → 1 hit (the placeholder)
5. The audit JSON IS the verification artifact, committed alongside the doc changes

No build runs needed. No pre-push hook changes (those are Group A-prime).

Order of operations

1. Update root `CLAUDE.md` — count bump + clause-5 expansion + new §6.N + §6.N-debt entry
2. Update root `AGENTS.md` — §6.N bullet
3. Update `lava-api-go/CLAUDE.md`, `AGENTS.md`, `CONSTITUTION.md` — §6.N reference blocks
4. Append §6.N reference block to all 16 `submodules/*/CLAUDE.md` (Containers gets stronger variant)
5. Write `.lava-ci-evidence/bluff-hunt/2026-05-05.json`
6. Write `.lava-ci-evidence/anti-bluff-audit-2026-05-05.json`
7. Commit parent + push to all 4 upstreams (`origin` multi-push)
8. Per-submodule (16): commit on `lava-pin/2026-05-05-clause-6n` branch (mirrors the §6.M `lava-pin/2026-05-04-clause-6m` pattern); push to each submodule's configured remotes
9. Bump parent submodule pointers; commit + push parent

Estimated duration: 30–45 minutes. No build/test runs.

Dependencies

- None within this spec (no other in-flight work blocks it)
- Forward dependency: Group A-prime spec is spawned by §6.N.3's debt entry. Group A-prime will brainstorm separately and produce its own implementation plan covering the pre-push hook enforcement (B+C of 6.N.1).
- This spec does NOT block any other work group (B–G from the 2026-05-05 multi-group brainstorm). They can proceed in parallel after Group A lands.

Open questions

None. All clarifying questions Q1–Q6 from the brainstorm session were answered:

- Q1: D (all three of cadence-tightening directions)
- Q2: D (all three of production-code-scope directions)
- Q3: D (all three of cadence-trigger directions)
- Q4: A (keep doc-only; spin off Group A-prime)
- Q5: C (full propagation matching §6.M pattern)
- Q6: 1 (single new clause + Seventh Law clause 5 expansion)
- Q7: "all good" (design approved)

Hand-off

After spec self-review + user approval, this spec is handed to the `writing-plans` skill to produce the implementation plan. The plan will track each of the 9 order-of-operations steps as a discrete task.